Machine-Level Programming II: Lab Session

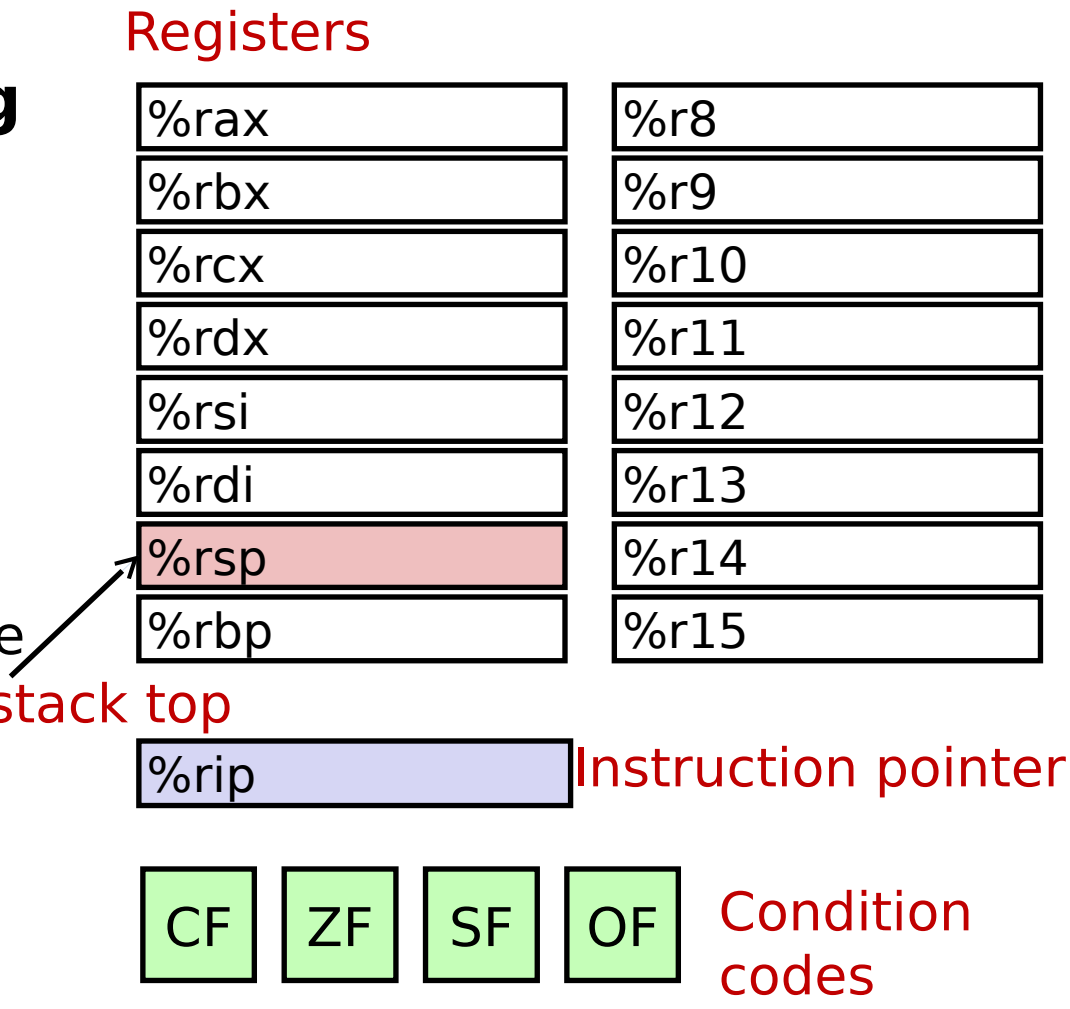
CSE 238/2038/2138: Systems Programming

Instructor:
Birol Gençyılmaz & Zuhal Altuntaş

Slides adapted from Bryant & O'Hallaron's slides

Processor State (x86-64, Partial)

- **Information about currently executing program**
- Temporary data
  - (%rax, …)
  - Location of runtime stack
  - (%rsp)
  - Location of current code control point
  - (%rip, …)
  - Status of most recent operations
  - (CF, ZF, SF, OF)



Condition Codes (Implicit Setting)

- **Single bit registers**

CF Carry Flag (for unsigned)
SF Sign Flag (for signed)
ZF Zero Flag
OF Overflow Flag (for signed)

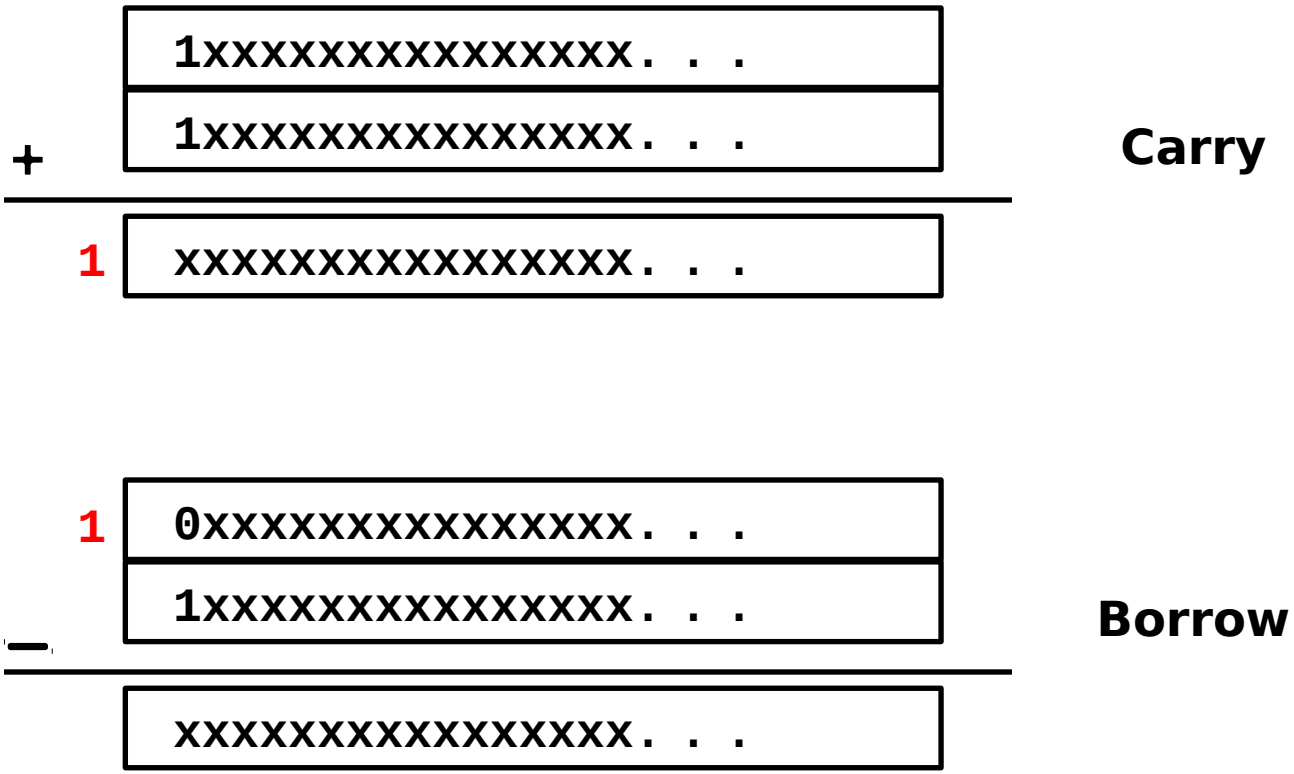
- **Implicitly set (think of it as a side effect) by arithmetic operations**

Example: addq Src,Dest ↔ t = a+b

  - CF set if carry out from most significant bit (unsigned overflow)
  - ZF set if t == 0
  - SF set if t < 0 (as signed)
  - OF set if two's-complement (signed) overflow
  - (a>0 && b>0 && t<0) || (a<0 && b<0 && t>=0)

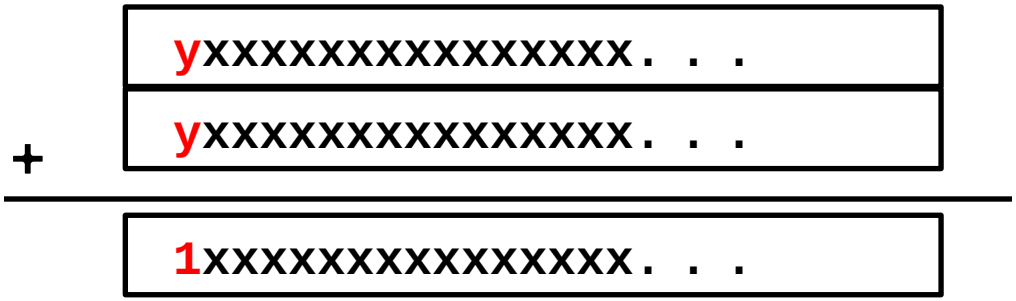
- **Not set by leaq instruction**

CF set when



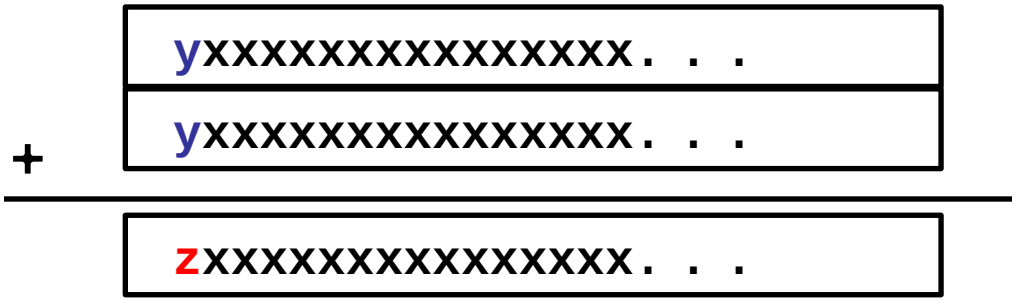
For unsigned arithmetic, this reports overflow

SF set when



For signed arithmetic, this reports when result is a negative number

OF set when



z = ~y

For signed arithmetic, this reports overflow

ZF set when

```
0000000000000000. . .000000
```

Condition Codes (Explicit Setting: Compare)

- **Explicit Setting by Compare Instruction**
  - cmpq Src2, Src1
  - Example: cmpq b,a like computing a-b without setting destination
  - CF set if carry out from most significant bit
    - (used for unsigned comparisons)
    - ZF set if a == b
    - SF set if (a-b) < 0 (as signed)
    - OF set if two's-complement (signed) overflow
    - (a>0 && b<0 && (a-b)<0) || (a<0 && b>0 && (a-b)>0)

Condition Codes (Explicit Setting: Test)

- **Explicit Setting by Test instruction**
  - testq Src2, Src1
  - Sets condition codes based on value of Src1 & Src2
  - Useful to have one of the operands be a mask

  - **Example:** testq b,a like computing a&b without setting destination
    - ZF set when a&b == 0
    - SF set when a&b < 0

Very often:

testq %rax,%rax

Reading Condition Codes

- **SetX Instructions**
  - Set low-order byte of destination to 0 or 1 based on combinations of condition codes
  - Does not alter remaining 7 bytes

SetX	Condition	Description
sete	ZF	Equal / Zero
setne	~ZF	Not Equal / Not Zero
sets	SF	Negative
setns	~SF	Nonnegative
setg	~(SF^OF)&~ZF	Greater (Signed)
setge	~(SF^OF)	Greater or Equal (Signed)
setl	(SF^OF)	Less (Signed)
setle	(SF^OF)|ZF	Less or Equal (Signed)
seta	~CF&~ZF	Above (unsigned)
setb	CF	Below (unsigned)

Exercise

SetX	Condition	Description
sete	ZF	Equal / Zero
setne	~ZF	Not Equal / Not Zero
sets	SF	Negative
setns	~SF	Nonnegative
setg	~(SF^OF)&~ZF	Greater (Signed)
setge	~(SF^OF)	Greater or Equal (Signed)
setl	(SF^OF)	Less (Signed)
setle	(SF^OF)|ZF	Less or Equal (Signed)
seta	~CF&~ZF	Above (unsigned)
setb	CF	Below (unsigned)

- cmpq b,a :like computing a-b

without setting destination

CF set if carry out from most significant bit (used for unsigned comparisons)
ZF set if a == b
SF set if (a-b) < 0 (as signed)
OF set if two's-complement (signed) overflow
(a>0 && b<0 && (a-b)<0) || (a<0 && b>0 && (a-b)>0)

```
xor    %rax, %rax
sub    $1, %rax
cmp    $2, %rax
setl   %al
movzbl %al, %eax
```

%rax	SF	CF	OF	ZF
0x 0000 0000 0000 0000	0	0	0	1
0x FFFF FFFF FFFF FFFF	1	1	0	0
0x FFFF FFFF FFFF FFFF	1	0	0	0
0x FFFF FFFF FFFF FF01	1	0	0	0
0x 0000 0000 0000 0001	1	0	0	0

Jumping

- **jX Instructions**
  - Jump to different part of code depending on condition codes

jX	Condition	Description
jmp	1	Unconditional
je	ZF	Equal / Zero
jne	~ZF	Not Equal / Not Zero
js	SF	Negative
jns	~SF	Nonnegative
jg	~(SF^OF)&~ZF	Greater (Signed)
jge	~(SF^OF)	Greater or Equal (Signed)
jl	(SF^OF)	Less (Signed)
jle	(SF^OF)|ZF	Less or Equal (Signed)
ja	~CF&~ZF	Above (unsigned)
jb	CF	Below (unsigned)

Conditional Branch Example (Old Style)

- Generation

```
unix> gcc –Og -S –fno-if-conversion control.c
```

```
long absdiff
  (long x, long y)
{
  long result;
  if (x > y)
    result = x-y;
  else
    result = y-x;
  return result;
}
```

```
absdiff:
   cmpq     %rsi, %rdi  # x:y
   jle      .L4
   movq     %rdi, %rax
   subq     %rsi, %rax
   ret
.L4:                    # x <= y
   movq     %rsi, %rax
   subq     %rdi, %rax
   ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value

Expressing with Goto Code

- C allows **goto** statement
- Jump to position designated by label

```
long absdiff
  (long x, long y)
{
    long result;
    if (x > y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```

```
long absdiff_j
  (long x, long y)
{
    long result;
    int nottest = x <= y;
    if (nottest) goto Else;
    result = x-y;
    goto Done;
 Else:
    result = y-x;
 Done:
    return result;
}
```

```
absdiff:
   cmpq    %rsi, %rdi   # x:y
   jle     .L4
   movq    %rdi, %rax
   subq    %rsi, %rax
   ret
.L4:                    # x <= y
   movq    %rsi, %rax
   subq    %rdi, %rax
   ret
```

Conditional Move

- **cmovX Instructions**
  - Copy the value when the move condition holds

cmovX	Condition	Description
cmove	ZF	Equal / Zero
cmovne	~ZF	Not Equal / Not Zero
cmovs	SF	Negative
cmovns	~SF	Nonnegative
cmovg	~(SF^OF)&~ZF	Greater (Signed >)
cmovge	~(SF^OF)	Greater or Equal (Signed >=)
cmovl	(SF^OF)	Less (Signed <)
cmovle	(SF^OF)|ZF	Less or Equal (Signed <=)
cmova	~CF&~ZF	Above (unsigned >)
cmovae	~CF	Above or Equal (unsigned >=)
cmovb	CF	Below (unsigned <)
cmovbe	CF|ZF	Below or Equal (unsigned <=)

Conditional Move Example

```
long absdiff
  (long x, long y)
{
    long result;
    if (x > y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```

```
  result = x-y;
eval = y-x;
nottest = (x <= y);
if (nottest) result = eval;
return result;
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value

```
absdiff:
    movq    %rdi, %rax  # x
    subq    %rsi, %rax  # result = x-y
    movq    %rsi, %rdx  # y
    subq    %rdi, %rdx  # eval = y-x
    cmpq    %rsi, %rdi  # x:y
    cmovle  %rdx, %rax  # if <=, result = eval
    ret
```

“Do-While” Loop Example

C Code

```
long pcount_do
  (unsigned long x) {
  long result = 0;
  do {
    result += x & 0x1;
    x >>= 1;
  } while (x);
  return result;
}
```

Goto Version

```
long pcount_goto
  (unsigned long x) {
  long result = 0;
 loop:
  result += x & 0x1;
  x >>= 1;
  if(x) goto loop;
  return result;
}
```

- **Count number of 1’s in argument** x **(“popcount”)**
- **Use conditional branch**
- **to either continue looping or to exit loop**

“Do-While” Loop Compilation

Goto Version

```
long pcount_goto
  (unsigned long x) {
  long result = 0;
 loop:
  result += x & 0x1;
  x >>= 1;
  if(x) goto loop;
  return result;
}
```

Register	Use(s)
%rdi	Argument x
%rax	result

```
    movl    $0, %eax        #  result = 0
.L2:                        #  loop:
   movq    %rdi, %rdx
   andl    $1, %edx         #  t = x & 0x1
   addq    %rdx, %rax       #  result += t
   shrq    %rdi             #  x >>= 1
   jne     .L2              #  if (x) goto loop
   rep; ret
```

General “While” Translation #1

While version

```
while (Test)
  Body
```



Goto Version #1

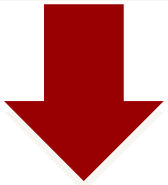
```
  goto test;
loop:
  Body
test:
  if (Test)
    goto loop;
done:
```

Used with
-Og

General “While” Translation #2

While version

```
while (Test)
  Body
```



Do-While Version

```
  if (!Test)
    goto done;
  do
    Body
  while(Test);
done:
```



Used with
-01

Goto Version #2

```
  if (!Test)
    goto done;
loop:
  Body
  if (Test)
    goto loop;
done:
```

“For” Loop While Loop

For Version

```
for (Init; Test; Update)
        Body
```



While Version

```
Init;
while (Test) {
    Body
    Update;
}
```

```
long switch_eg
   (long x, long y, long z)
{
    long w = 1;
    switch(x) {
    case 1:
        w = y*z;
        break;
    case 2:
        w = y/z;
        /* Fall Through */
    case 3:
        w += z;
        break;
    case 5:
    case 6:
        w -= z;
        break;
    default:
        w = 2;
    }
    return w;
}
```

Switch Statement Example

- Multiple case labels
  - Here: 5 & 6
- Fall through cases
  - Here: 2
- Missing cases
  - Here: 4

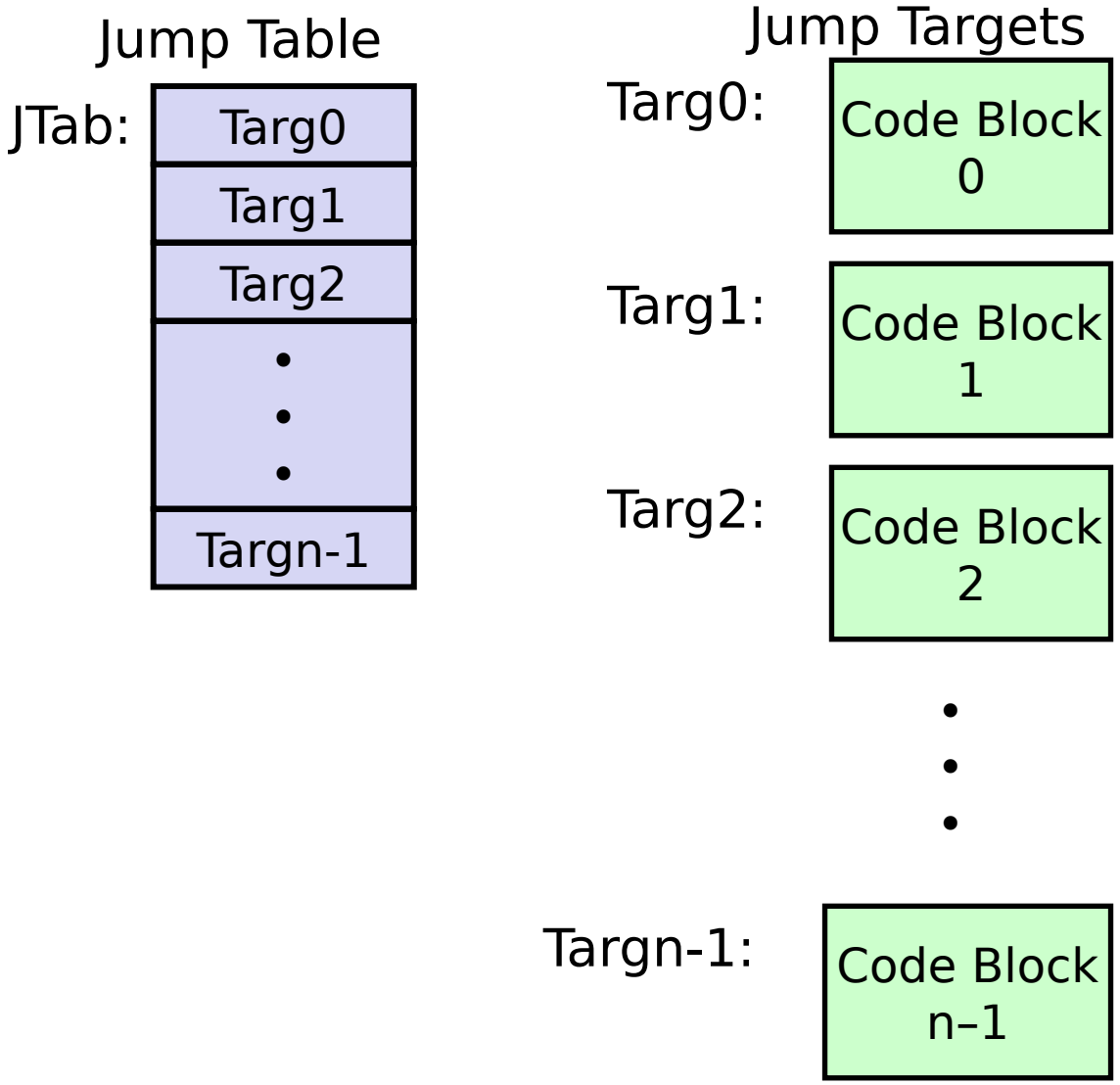
Jump Table Structure

Switch Form

```
switch(x) {
  case val_0:
    Block 0
  case val_1:
    Block 1
    • • •
  case val_n-1:
    Block n–1
}
```

Translation (Extended C)

```
goto *JTab[x];
```



Exercise 1
Practise Problem 3.16

When given the C code

```
void cond(short a, short *p)
{
    if (a && *p < a)
        *p = a;
}
```

GCC generates the following assembly code:

```
  void cond(short a, short *p)
  a in %rdi, p in %rsi
cond:
  testq   %rdi, %rdi
  je      .L1
  cmpq    %rsi, (%rdi)
  jle     .L1
  movq    %rdi, (%rsi)
.L1:
  rep; ret
```

A. Write a goto version in C that performs the same computation and mimics the control flow of the assembly code, in the style shown in Figure 3.16(b). You might find it helpful to first annotate the assembly code as we have done in our examples.

B. Explain why the assembly code contains two conditional branches, even though the C code has only one `if` statement.

Exercise 1

Practise Problem 3.16

A. Write a goto version in C that performs the same computation and mimics the control flow of the assembly code, in the style shown in Figure 3.16(b). You might find it helpful to first annotate the assembly code as we have done in our examples.

```
void cond(short a, short *p)
{
    if (a && *p < a)
        *p = a;
}
```

```
  void cond(short a, short *p)
  a in %rdi, p in %rsi
cond:
  testq   %rdi, %rdi
  je      .L1
  cmpq    %rsi, (%rdi)
  jle     .L1
  movq    %rdi, (%rsi)
.L1:
  rep; ret
```

```
void cond (short a, short *p) {
    if (a == 0)
        goto done;
    if (a >= *p)
        goto done;
    *p = a;
    done:
        return;
}
```

Exercise 1
Practise Problem 3.16

B. Explain why the assembly code contains two conditional branches, even though the C code has only one `if` statement.

```
void cond(short a, short *p)
{
    if (a && *p < a)
        *p = a;
}
```

```
  void cond(short a, short *p)
  a in %rdi, p in %rsi
cond:
  testq   %rdi, %rdi
  je      .L1
  cmpq    %rsi, (%rdi)
  jle     .L1
  movq    %rdi, (%rsi)
.L1:
  rep; ret
```

If a is false (meaning, if a is zero), then the code directly jumps to the label L1 and rest of the calculations are not done!

Exercise 2
Practise Problem 3.18

Starting with C code of the form

```
short test(short x, short y, short z) {
    short val = __________;
    if (__________) {
        if (__________)
            val = __________;
        else
            val = __________;
    } else if (__________)
        val = __________;
    return val;
}
```

GCC generates the following assembly code:

```
  short test(short x, short y, short z)
  x in %rdi, y in %rsi, z in %rdx
test:
  leaq    (%rdx,%rsi), %rax
  subq    %rdi, %rax
  cmpq    $5, %rdx
  jle     .L2
  cmpq    $2, %rsi
  jle     .L3
  movq    %rdi, %rax
  idivq   %rdx, %rax
  ret
.L3:
  movq    %rdi, %rax
  idivq   %rsi, %rax
  ret
.L2:
  cmpq    $3, %rdx
  jge     .L4
  movq    %rdx, %rax
  idivq   %rsi, %rax
.L4:
  rep; ret
```

Fill in the missing expressions in the C code.

Exercise 2
Practise Problem 3.18

Starting with C code of the form

```
short test(short x, short y, short z) {
    short val = y + z - x ;
    if (_____) {
        if (_____)
            val = _____;
        else
            val = _____;
    } else if (_____)
        val = _____;
    return val;
}
```

GCC generates the following assembly code:

```
  short test(short x, short y, short z)
  x in %rdi, y in %rsi, z in %rdx
test:
  leaq    (%rdx,%rsi), %rax
  subq    %rdi, %rax
  cmpq    $5, %rdx
  jle     .L2
  cmpq    $2, %rsi
  jle     .L3
  movq    %rdi, %rax
  idivq   %rdx, %rax
  ret
.L3:
  movq    %rdi, %rax
  idivq   %rsi, %rax
  ret
.L2:
  cmpq    $3, %rdx
  jge     .L4
  movq    %rdx, %rax
  idivq   %rsi, %rax
.L4:
  rep; ret
```

Fill in the missing expressions in the C code.

Exercise 2
Practise Problem 3.18

Starting with C code of the form

```
short test(short x, short y, short z) {
    short val = y + z - x ;
    if ( z > 5 ) {
        if (_____)
            val = _____;
        else
            val = _____;
    } else if (_____)
        val = _____;
    return val;
}
```

GCC generates the following assembly code:

```
  short test(short x, short y, short z)
  x in %rdi, y in %rsi, z in %rdx
test:
  leaq    (%rdx,%rsi), %rax
  subq    %rdi, %rax
  cmpq    $5, %rdx
  jle     .L2
  cmpq    $2, %rsi
  jle     .L3
  movq    %rdi, %rax
  idivq   %rdx, %rax
  ret
.L3:
  movq    %rdi, %rax
  idivq   %rsi, %rax
  ret
.L2:
  cmpq    $3, %rdx
  jge     .L4
  movq    %rdx, %rax
  idivq   %rsi, %rax
.L4:
  rep; ret
```

Fill in the missing expressions in the C code.

Exercise 2
Practise Problem 3.18

Starting with C code of the form

```
short test(short x, short y, short z) {
    short val = y + z - x ;
    if ( z > 5 ) {
        if ( y > 2 )
            val = _____;
        else
            val = _____;
    } else if (_____)
        val = _____;
    return val;
}
```

GCC generates the following assembly code:

```
  short test(short x, short y, short z)
  x in %rdi, y in %rsi, z in %rdx
test:
  leaq    (%rdx,%rsi), %rax
  subq    %rdi, %rax
  cmpq    $5, %rdx
  jle     .L2
  cmpq    $2, %rsi
  jle     .L3
  movq    %rdi, %rax
  idivq   %rdx, %rax
  ret
.L3:
  movq    %rdi, %rax
  idivq   %rsi, %rax
  ret
.L2:
  cmpq    $3, %rdx
  jge     .L4
  movq    %rdx, %rax
  idivq   %rsi, %rax
.L4:
  rep; ret
```

Fill in the missing expressions in the C code.

Exercise 2
Practise Problem 3.18

Starting with C code of the form

```
short test(short x, short y, short z) {
    short val = y + z - x ;
    if ( z > 5 ) {
        if ( y > 2 )
            val = x / z ;
        else
            val = _____ ;
    } else if ( _____ )
        val = _____ ;
    return val;
}
```

GCC generates the following assembly code:

```
  short test(short x, short y, short z)
  x in %rdi, y in %rsi, z in %rdx
test:
  leaq    (%rdx,%rsi), %rax
  subq    %rdi, %rax
  cmpq    $5, %rdx
  jle     .L2
  cmpq    $2, %rsi
  jle     .L3
  movq    %rdi, %rax
  idivq   %rdx, %rax
  ret
.L3:
  movq    %rdi, %rax
  idivq   %rsi, %rax
  ret
.L2:
  cmpq    $3, %rdx
  jge     .L4
  movq    %rdx, %rax
  idivq   %rsi, %rax
.L4:
  rep; ret
```

Fill in the missing expressions in the C code.

Exercise 2
Practise Problem 3.18

Starting with C code of the form

```
short test(short x, short y, short z) {
    short val = y + z - x ;
    if ( z > 5 ) {
        if ( y > 2 )
            val = x / z ;
        else
            val = x / y ;
    } else if (          )
        val =          ;
    return val;
}
```

GCC generates the following assembly code:

```
  short test(short x, short y, short z)
  x in %rdi, y in %rsi, z in %rdx
test:
  leaq    (%rdx,%rsi), %rax
  subq    %rdi, %rax
  cmpq    $5, %rdx
  jle     .L2
  cmpq    $2, %rsi
  jle     .L3
  movq    %rdi, %rax
  idivq   %rdx, %rax
  ret
.L3:
  movq    %rdi, %rax
  idivq   %rsi, %rax
  ret
.L2:
  cmpq    $3, %rdx
  jge     .L4
  movq    %rdx, %rax
  idivq   %rsi, %rax
.L4:
  rep; ret
```

Fill in the missing expressions in the C code.

Exercise 2
Practise Problem 3.18

Starting with C code of the form

```
short test(short x, short y, short z) {
    short val = y + z - x ;
    if ( z > 5 ) {
        if ( y > 2 )
            val = x / z ;
        else
            val = x / y ;
    } else if ( z < 3 )
        val = _____ ;
    return val;
}
```

GCC generates the following assembly code:

```
  short test(short x, short y, short z)
  x in %rdi, y in %rsi, z in %rdx
test:
  leaq    (%rdx,%rsi), %rax
  subq    %rdi, %rax
  cmpq    $5, %rdx
  jle     .L2
  cmpq    $2, %rsi
  jle     .L3
  movq    %rdi, %rax
  idivq   %rdx, %rax
  ret
.L3:
  movq    %rdi, %rax
  idivq   %rsi, %rax
  ret
.L2:
  cmpq    $3, %rdx
  jge     .L4
  movq    %rdx, %rax
  idivq   %rsi, %rax
.L4:
  rep; ret
```

Fill in the missing expressions in the C code.

Exercise 2
Practise Problem 3.18

Starting with C code of the form

```
short test(short x, short y, short z) {
    short val = y + z - x ;
    if ( z > 5 ) {
        if ( y > 2 )
            val = x / z ;
        else
            val = x / y ;
    } else if ( z < 3 )
        val = z / y ;
    return val;
}
```

GCC generates the following assembly code:

```
  short test(short x, short y, short z)
  x in %rdi, y in %rsi, z in %rdx
test:
  leaq    (%rdx,%rsi), %rax
  subq    %rdi, %rax
  cmpq    $5, %rdx
  jle     .L2
  cmpq    $2, %rsi
  jle     .L3
  movq    %rdi, %rax
  idivq   %rdx, %rax
  ret
.L3:
  movq    %rdi, %rax
  idivq   %rsi, %rax
  ret
.L2:
  cmpq    $3, %rdx
  jge     .L4
  movq    %rdx, %rax
  idivq   %rsi, %rax
.L4:
  rep; ret
```

Fill in the missing expressions in the C code.

Exercise 3
Practise Problem 3.24

For C code having the general form

```
short loop_while(short a, short b)
{
    short result = _____;
    while (_____) {
        result = _____;
        a = _____;
    }
    return result;
}
```

```
short loop_while(short a, short b)
a in %rdi, b in %rsi
1   loop_while:
2     movl    $0, %eax
3     jmp     .L2
4   .L3:
5     leaq    (,%rsi,%rdi), %rdx
6     addq    %rdx, %rax
7     subq    $1, %rdi
8   .L2:
9     cmpq    %rsi, %rdi
10    jg      .L3
11    rep; ret
```

We can see that the compiler used a jump-to-middle translation, using the `jmp` instruction on line 3 to jump to the test starting with label `.L2`. Fill in the missing parts of the C code.

Exercise 3
Practise Problem 3.24

For C code having the general form

```
short loop_while(short a, short b)
{
    short result = ____0____;
    while (_________) {
        result = _________;
        a = _________;
    }
    return result;
}
```

```
short loop_while(short a, short b)
a in %rdi, b in %rsi
1   loop_while:
2     movl    $0, %eax
3     jmp     .L2
4   .L3:
5     leaq    (,%rsi,%rdi), %rdx
6     addq    %rdx, %rax
7     subq    $1, %rdi
8   .L2:
9     cmpq    %rsi, %rdi
10    jg      .L3
11    rep; ret
```

We can see that the compiler used a jump-to-middle translation, using the `jmp` instruction on line 3 to jump to the test starting with label `.L2`. Fill in the missing parts of the C code.

Exercise 3
Practise Problem 3.24

For C code having the general form

```
short loop_while(short a, short b)
{
    short result = ____0____;
    while ( __a > b__ ) {
        result = _________;
        a = _________;
    }
    return result;
}
```

```
short loop_while(short a, short b)
a in %rdi, b in %rsi
1   loop_while:
2     movl    $0, %eax
3     jmp     .L2
4   .L3:
5     leaq    (,%rsi,%rdi), %rdx
6     addq    %rdx, %rax
7     subq    $1, %rdi
8   .L2:
9     cmpq    %rsi, %rdi
10    jg      .L3
11    rep; ret
```

We can see that the compiler used a jump-to-middle translation, using the `jmp` instruction on line 3 to jump to the test starting with label `.L2`. Fill in the missing parts of the C code.

Exercise 3
Practise Problem 3.24

For C code having the general form

```
short loop_while(short a, short b)
{
    short result = ____0____;
    while ( ___a > b___ ) {
        result = ___a * b + result___;
        a = _____________;
    }
    return result;
}
```

```
short loop_while(short a, short b)
a in %rdi, b in %rsi
1   loop_while:
2     movl    $0, %eax
3     jmp     .L2
4   .L3:
5     leaq    (,%rsi,%rdi), %rdx
6     addq    %rdx, %rax
7     subq    $1, %rdi
8   .L2:
9     cmpq    %rsi, %rdi
10    jg      .L3
11    rep; ret
```

We can see that the compiler used a jump-to-middle translation, using the `jmp` instruction on line 3 to jump to the test starting with label `.L2`. Fill in the missing parts of the C code.

Exercise 3
Practise Problem 3.24

For C code having the general form

```
short loop_while(short a, short b)
{
    short result = 0;
    while ( a > b ) {
        result = a * b + result;
        a = a - 1;
    }
    return result;
}
```

```
short loop_while(short a, short b)
a in %rdi, b in %rsi
1   loop_while:
2     movl    $0, %eax
3     jmp     .L2
4   .L3:
5     leaq    (,%rsi,%rdi), %rdx
6     addq    %rdx, %rax
7     subq    $1, %rdi
8   .L2:
9     cmpq    %rsi, %rdi
10    jg      .L3
11    rep; ret
```

We can see that the compiler used a jump-to-middle translation, using the `jmp` instruction on line 3 to jump to the test starting with label `.L2`. Fill in the missing parts of the C code.

Exercise 4
Practise Problem 3.25

For C code having the general form

```
long loop_while2(long a, long b)
{
    long result = _____;
    while (_____) {
        result = _____;
        b = _____;
    }
    return result;
}
```

```
    a in %rdi, b in %rsi
1   loop_while2:
2     testq   %rsi, %rsi
3     jle     .L8
4     movq    %rsi, %rax
5   .L7:
6     imulq   %rdi, %rax
7     subq    %rdi, %rsi
8     testq   %rsi, %rsi
9     jg      .L7
10    rep; ret
11  .L8:
12    movq    %rsi, %rax
13    ret
```

We can see that the compiler used a guarded-do translation, using the `jle` instruction on line 3 to skip over the loop code when the initial test fails. Fill in the missing parts of the C code. Note that the control structure in the assembly

Exercise 4
Practise Problem 3.25

For C code having the general form

```
long loop_while2(long a, long b)
{
    long result = _____;
    while ( b > 0 ) {
        result = _____;
        b = _____;
    }
    return result;
}
```

```
    a in %rdi, b in %rsi
1   loop_while2:
2     testq   %rsi, %rsi
3     jle     .L8
4     movq    %rsi, %rax
5   .L7:
6     imulq   %rdi, %rax
7     subq    %rdi, %rsi
8     testq   %rsi, %rsi
9     jg      .L7
10    rep; ret
11  .L8:
12    movq    %rsi, %rax
13    ret
```

We can see that the compiler used a guarded-do translation, using the `jle` instruction on line 3 to skip over the loop code when the initial test fails. Fill in the missing parts of the C code. Note that the control structure in the assembly

Exercise 4
Practise Problem 3.25

For C code having the general form

```
long loop_while2(long a, long b)
{
    long result = ____b____;
    while ( ___b > 0___ ) {
        result = _________;
        b = _________;
    }
    return result;
}
```

```
    a in %rdi, b in %rsi
1   loop_while2:
2     testq   %rsi, %rsi
3     jle     .L8
4     movq    %rsi, %rax
5   .L7:
6     imulq   %rdi, %rax
7     subq    %rdi, %rsi
8     testq   %rsi, %rsi
9     jg      .L7
10    rep; ret
11  .L8:
12    movq    %rsi, %rax
13    ret
```

We can see that the compiler used a guarded-do translation, using the `jle` instruction on line 3 to skip over the loop code when the initial test fails. Fill in the missing parts of the C code. Note that the control structure in the assembly

Exercise 4
Practise Problem 3.25

For C code having the general form

```
long loop_while2(long a, long b)
{
    long result = ____b____;
    while ( ___b > 0___ ) {
        result = a * result ;
        b = _________;
    }
    return result;
}
```

```
    a in %rdi, b in %rsi
1   loop_while2:
2     testq   %rsi, %rsi
3     jle     .L8
4     movq    %rsi, %rax
5   .L7:
6     imulq   %rdi, %rax
7     subq    %rdi, %rsi
8     testq   %rsi, %rsi
9     jg      .L7
10    rep; ret
11  .L8:
12    movq    %rsi, %rax
13    ret
```

We can see that the compiler used a guarded-do translation, using the `jle` instruction on line 3 to skip over the loop code when the initial test fails. Fill in the missing parts of the C code. Note that the control structure in the assembly

Exercise 4
Practise Problem 3.25

For C code having the general form

```
long loop_while2(long a, long b)
{
    long result = ____b____;
    while ( ___b > 0___ ) {
        result = a * result;
        b = ____b - a____;
    }
    return result;
}
```

```
    a in %rdi, b in %rsi
1   loop_while2:
2     testq   %rsi, %rsi
3     jle     .L8
4     movq    %rsi, %rax
5   .L7:
6     imulq   %rdi, %rax
7     subq    %rdi, %rsi
8     testq   %rsi, %rsi
9     jg      .L7
10    rep; ret
11  .L8:
12    movq    %rsi, %rax
13    ret
```

We can see that the compiler used a guarded-do translation, using the `jle` instruction on line 3 to skip over the loop code when the initial test fails. Fill in the missing parts of the C code. Note that the control structure in the assembly

Exercise 5
Practise Problem 3.31

For a C function `switcher` with the general structure

```
void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
    case __________:        /* Case A */
        c = __________;
        /* Fall through */
    case __________:        /* Case B */
        val = __________;
        break;
    case __________:        /* Case C */
    case __________:        /* Case D */
        val = __________;
        break;
    case __________:        /* Case E */
        val = __________;
        break;
    default:
        val = __________;
    }
    *dest = val;
}
```

```
void switcher(long a, long b, long c, long *dest)
a in %rsi, b in %rdi, c in %rdx, d in %rcx
1   switcher:
2     cmpq    $7, %rdi
3     ja      .L2
4     jmp     *.L4(,%rdi,8)
5     .section        .rodata
6   .L7:
7     xorq    $15, %rsi
8     movq    %rsi, %rdx
9   .L3:
10    leaq    112(%rdx), %rdi
11    jmp     .L6
12  .L5:
13    leaq    (%rdx,%rsi), %rdi
14    salq    $2, %rdi
15    jmp     .L6
16  .L2:
17    movq    %rsi, %rdi
18  .L6:
19    movq    %rdi, (%rcx)
20    ret
```

```
1   .L4:
2     .quad   .L3
3     .quad   .L2
4     .quad   .L5
5     .quad   .L2
6     .quad   .L6
7     .quad   .L7
8     .quad   .L2
9     .quad   .L5
```

GCC generates the assembly code and jump table shown in Figure 3.24.

Fill in the missing parts of the C code. Except for the ordering of case labels C and D, there is only one way to fit the different cases into the template.

Exercise 5
Practise Problem 3.31

For a C function `switcher` with the general structure

```
void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
    case _________:       /* Case A */
        c = _________;
        /* Fall through */
    case _________:       /* Case B */
        val = _________;
        break;
    case _________:       /* Case C */
    case _________:       /* Case D */
        val = _________;
        break;
    case _________:       /* Case E */
        val = _________;
        break;
    default:
        val = __a_____;
    }
    *dest = val;
}
```

GCC generates the assembly code and jump table shown in Figure 3.24.

Fill in the missing parts of the C code. Except for the ordering of case labels C and D, there is only one way to fit the different cases into the template.

```
void switcher(long a, long b, long c, long *dest)
a in %rsi, b in %rdi, c in %rdx, d in %rcx
1   switcher:
2     cmpq    $7, %rdi
3     ja      .L2
4     jmp     *.L4(,%rdi,8)
5     .section        .rodata
6   .L7:
7     xorq    $15, %rsi
8     movq    %rsi, %rdx
9   .L3:
10    leaq    112(%rdx), %rdi
11    jmp     .L6
12  .L5:
13    leaq    (%rdx,%rsi), %rdi
14    salq    $2, %rdi
15    jmp     .L6
16  .L2:
17    movq    %rsi, %rdi
18  .L6:
19    movq    %rdi, (%rcx)
20    ret
```

```
1   .L4:
2     .quad   .L3
3     .quad   .L2
4     .quad   .L5
5     .quad   .L2
6     .quad   .L6
7     .quad   .L7
8     .quad   .L2
9     .quad   .L5
```

Exercise 5
Practise Problem 3.31

For a C function `switcher` with the general structure

```
void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
    case _________:        /* Case A */
        c = _________;
        /* Fall through */
    case _________:        /* Case B */
        val = _________;
        break;
    case ____2____:        /* Case C */
    case ____7____:        /* Case D */
        val = _________;
        break;
    case _________:        /* Case E */
        val = _________;
        break;
    default:
        val = ____a____;
    }
    *dest = val;
}
```

GCC generates the assembly code and jump table shown in Figure 3.24.

Fill in the missing parts of the C code. Except for the ordering of case labels C and D, there is only one way to fit the different cases into the template.

```
1   void switcher(long a, long b, long c, long *dest)
    a in %rsi, b in %rdi, c in %rdx, d in %rcx
1   switcher:
2     cmpq    $7, %rdi
3     ja      .L2
4     jmp     *.L4(,%rdi,8)
5     .section        .rodata
6   .L7:
7     xorq    $15, %rsi
8     movq    %rsi, %rdx
9   .L3:
10    leaq    112(%rdx), %rdi
11    jmp     .L6
12  .L5:
13    leaq    (%rdx,%rsi), %rdi
14    salq    $2, %rdi
15    jmp     .L6
16  .L2:
17    movq    %rsi, %rdi
18  .L6:
19    movq    %rdi, (%rcx)
20    ret
```

```
1   .L4:
2     .quad   .L3
3     .quad   .L2
4     .quad   .L5
5     .quad   .L2
6     .quad   .L6
7     .quad   .L7
8     .quad   .L2
9     .quad   .L5
```

Exercise 5
Practise Problem 3.31

For a C function `switcher` with the general structure

```
void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
    case _________:       /* Case A */
        c = _________;
        /* Fall through */
    case _________:       /* Case B */
        val = _________;
        break;
    case ____2____:       /* Case C */
    case ____7____:       /* Case D */
        val = (c + a) << 2;
        break;
    case _________:       /* Case E */
        val = _________;
        break;
    default:
        val = ___a____;
    }
    *dest = val;
}
```

GCC generates the assembly code and jump table shown in Figure 3.24.

Fill in the missing parts of the C code. Except for the ordering of case labels C and D, there is only one way to fit the different cases into the template.

```
void switcher(long a, long b, long c, long *dest)
a in %rsi, b in %rdi, c in %rdx, d in %rcx
1   switcher:
2     cmpq    $7, %rdi
3     ja      .L2
4     jmp     *.L4(,%rdi,8)
5     .section        .rodata
6   .L7:
7     xorq    $15, %rsi
8     movq    %rsi, %rdx
9   .L3:
10    leaq    112(%rdx), %rdi
11    jmp     .L6
12  .L5:
13    leaq    (%rdx,%rsi), %rdi
14    salq    $2, %rdi
15    jmp     .L6
16  .L2:
17    movq    %rsi, %rdi
18  .L6:
19    movq    %rdi, (%rcx)
20    ret
```

```
1   .L4:
2     .quad   .L3
3     .quad   .L2
4     .quad   .L5
5     .quad   .L2
6     .quad   .L6
7     .quad   .L7
8     .quad   .L2
9     .quad   .L5
```

Exercise 5
Practise Problem 3.31

For a C function `switcher` with the general structure

```
void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
    case ____5____:        /* Case A */
        c = _________;
        /* Fall through */
    case _________:        /* Case B */
        val = _________;
        break;
    case ____2____:        /* Case C */
    case ____7____:        /* Case D */
        val = (c + a) << 2;
        break;
    case _________:        /* Case E */
        val = _________;
        break;
    default:
        val = ____a____;
    }
    *dest = val;
}
```

GCC generates the assembly code and jump table shown in Figure 3.24.

Fill in the missing parts of the C code. Except for the ordering of case labels C and D, there is only one way to fit the different cases into the template.

```
1   void switcher(long a, long b, long c, long *dest)
    a in %rsi, b in %rdi, c in %rdx, d in %rcx
2   switcher:
3     cmpq    $7, %rdi
4     ja      .L2
5     jmp     *.L4(,%rdi,8)
6     .section        .rodata
7   .L7:
8     xorq    $15, %rsi
9     movq    %rsi, %rdx
10  .L3:
11    leaq    112(%rdx), %rdi
12    jmp     .L6
13  .L5:
14    leaq    (%rdx,%rsi), %rdi
15    salq    $2, %rdi
16    jmp     .L6
17  .L2:
18    movq    %rsi, %rdi
19  .L6:
20    movq    %rdi, (%rcx)
      ret
```

```
1   .L4:
2     .quad   .L3
3     .quad   .L2
4     .quad   .L5
5     .quad   .L2
6     .quad   .L6
7     .quad   .L7
8     .quad   .L2
9     .quad   .L5
```

Exercise 5
Practise Problem 3.31

For a C function `switcher` with the general structure

```
void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
    case ___5___:       /* Case A */
        c = a ^ 15 ;
        /* Fall through */
    case _________:     /* Case B */
        val = _________;
        break;
    case ___2___:       /* Case C */
    case ___7___:       /* Case D */
        val = (c + a) << 2;
        break;
    case _________:     /* Case E */
        val = _________;
        break;
    default:
        val = ___a___;
    }
    *dest = val;
}
```

GCC generates the assembly code and jump table shown in Figure 3.24.

Fill in the missing parts of the C code. Except for the ordering of case labels C and D, there is only one way to fit the different cases into the template.

```
void switcher(long a, long b, long c, long *dest)
a in %rsi, b in %rdi, c in %rdx, d in %rcx
1   switcher:
2     cmpq    $7, %rdi
3     ja      .L2
4     jmp     *.L4(,%rdi,8)
5     .section        .rodata
6   .L7:
7     xorq    $15, %rsi
8     movq    %rsi, %rdx
9   .L3:
10    leaq    112(%rdx), %rdi
11    jmp     .L6
12  .L5:
13    leaq    (%rdx,%rsi), %rdi
14    salq    $2, %rdi
15    jmp     .L6
16  .L2:
17    movq    %rsi, %rdi
18  .L6:
19    movq    %rdi, (%rcx)
20    ret
```

```
1   .L4:
2     .quad   .L3
3     .quad   .L2
4     .quad   .L5
5     .quad   .L2
6     .quad   .L6
7     .quad   .L7
8     .quad   .L2
9     .quad   .L5
```

Exercise 5
Practise Problem 3.31

For a C function `switcher` with the general structure

```
void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
    case   5    :       /* Case A */
        c = a ^ 15 ;
        /* Fall through */
    case   0    :       /* Case B */
        val = _____;
        break;
    case   2    :       /* Case C */
    case   7    :       /* Case D */
        val = (c + a) << 2;
        break;
    case _____  :       /* Case E */
        val = _____;
        break;
    default:
        val =  a    ;
    }
    *dest = val;
}
```

GCC generates the assembly code and jump table shown in Figure 3.24.

Fill in the missing parts of the C code. Except for the ordering of case labels C and D, there is only one way to fit the different cases into the template.

```
void switcher(long a, long b, long c, long *dest)
a in %rsi, b in %rdi, c in %rdx, d in %rcx
1   switcher:
2     cmpq    $7, %rdi
3     ja      .L2
4     jmp     *.L4(,%rdi,8)
5     .section        .rodata
6   .L7:
7     xorq    $15, %rsi
8     movq    %rsi, %rdx
9   .L3:
10    leaq    112(%rdx), %rdi
11    jmp     .L6
12  .L5:
13    leaq    (%rdx,%rsi), %rdi
14    salq    $2, %rdi
15    jmp     .L6
16  .L2:
17    movq    %rsi, %rdi
18  .L6:
19    movq    %rdi, (%rcx)
20    ret
```

```
1   .L4:
2     .quad   .L3
3     .quad   .L2
4     .quad   .L5
5     .quad   .L2
6     .quad   .L6
7     .quad   .L7
8     .quad   .L2
9     .quad   .L5
```

Exercise 5
Practise Problem 3.31

For a C function `switcher` with the general structure

```
void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
    case   5    :        /* Case A */
        c = a ^ 15 ;
        /* Fall through */
    case   0    :        /* Case B */
        val = c + 112 ;
        break;
    case   2    :        /* Case C */
    case   7    :        /* Case D */
        val = (c + a) << 2 ;
        break;
    case        :        /* Case E */
        val =       ;
        break;
    default:
        val =  a    ;
    }
    *dest = val;
}
```

GCC generates the assembly code and jump table shown in Figure 3.24.

Fill in the missing parts of the C code. Except for the ordering of case labels C and D, there is only one way to fit the different cases into the template.

```
void switcher(long a, long b, long c, long *dest)
a in %rsi, b in %rdi, c in %rdx, d in %rcx
1   switcher:
2     cmpq    $7, %rdi
3     ja      .L2
4     jmp     *.L4(,%rdi,8)
5     .section        .rodata
6   .L7:
7     xorq    $15, %rsi
8     movq    %rsi, %rdx
9   .L3:
10    leaq    112(%rdx), %rdi
11    jmp     .L6
12  .L5:
13    leaq    (%rdx,%rsi), %rdi
14    salq    $2, %rdi
15    jmp     .L6
16  .L2:
17    movq    %rsi, %rdi
18  .L6:
19    movq    %rdi, (%rcx)
20    ret
```

```
1   .L4:
2     .quad   .L3
3     .quad   .L2
4     .quad   .L5
5     .quad   .L2
6     .quad   .L6
7     .quad   .L7
8     .quad   .L2
9     .quad   .L5
```

Exercise 5
Practise Problem 3.31

For a C function `switcher` with the general structure

```
void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
    case   5    :        /* Case A */
        c = a ^ 15 ;
        /* Fall through */
    case   0    :        /* Case B */
        val = c + 112 ;
        break;
    case   2    :        /* Case C */
    case   7    :        /* Case D */
        val = (c + a) << 2 ;
        break;
    case   4    :        /* Case E */
        val =          ;
        break;
    default:
        val =  a    ;
    }
    *dest = val;
}
```

GCC generates the assembly code and jump table shown in Figure 3.24.

Fill in the missing parts of the C code. Except for the ordering of case labels C and D, there is only one way to fit the different cases into the template.

```
void switcher(long a, long b, long c, long *dest)
a in %rsi, b in %rdi, c in %rdx, d in %rcx
1   switcher:
2     cmpq    $7, %rdi
3     ja      .L2
4     jmp     *.L4(,%rdi,8)
5     .section        .rodata
6   .L7:
7     xorq    $15, %rsi
8     movq    %rsi, %rdx
9   .L3:
10    leaq    112(%rdx), %rdi
11    jmp     .L6
12  .L5:
13    leaq    (%rdx,%rsi), %rdi
14    salq    $2, %rdi
15    jmp     .L6
16  .L2:
17    movq    %rsi, %rdi
18  .L6:
19    movq    %rdi, (%rcx)
20    ret
```

```
1   .L4:
2     .quad   .L3
3     .quad   .L2
4     .quad   .L5
5     .quad   .L2
6     .quad   .L6
7     .quad   .L7
8     .quad   .L2
9     .quad   .L5
```

Exercise 5
Practise Problem 3.31

For a C function `switcher` with the general structure

```
void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
    case __5__:        /* Case A */
        c = __a ^ 15__;
        /* Fall through */
    case __0__:        /* Case B */
        val = __c + 112__;
        break;
    case __2__:        /* Case C */
    case __7__:        /* Case D */
        val = __(c + a) << 2__;
        break;
    case __4__:        /* Case E */
        val = __b__;
        break;
    default:
        val = __a__;
    }
    *dest = val;
}
```

GCC generates the assembly code and jump table shown in Figure 3.24.

Fill in the missing parts of the C code. Except for the ordering of case labels C and D, there is only one way to fit the different cases into the template.

```
1   void switcher(long a, long b, long c, long *dest)
    a in %rsi, b in %rdi, c in %rdx, d in %rcx
2   switcher:
3     cmpq    $7, %rdi
4     ja      .L2
5     jmp     *.L4(,%rdi,8)
6     .section        .rodata
7   .L7:
8     xorq    $15, %rsi
9     movq    %rsi, %rdx
10  .L3:
11    leaq    112(%rdx), %rdi
12    jmp     .L6
13  .L5:
14    leaq    (%rdx,%rsi), %rdi
15    salq    $2, %rdi
16    jmp     .L6
17  .L2:
18    movq    %rsi, %rdi
19  .L6:
20    movq    %rdi, (%rcx)
      ret
```

```
1   .L4:
2     .quad   .L3
3     .quad   .L2
4     .quad   .L5
5     .quad   .L2
6     .quad   .L6
7     .quad   .L7
8     .quad   .L2
9     .quad   .L5
```